



نام درس: شبکه های کامپیوتری
نویسنده: محمد امانلو - ۸۴۰۰۰۰۱۰۸۱



خلاصه شماره
۲۹

خلاصه و توضیح فصل ۶: Link Layer و LANs

ایده‌ی کلی این بخش این است که بفهمیم لایه‌ی پیوند داده یا همان Link layer دقیقاً چه کار می‌کند و چرا برای شبکه‌ی حیاتی است. هدف‌ها در این فصل بیشتر حول این می‌چرخد که سرویس‌های مهم Link layer را بشناسیم (مثل تشخیص و حتی گاهی اصلاح خطا)، بفهمیم وقتی چندتا دستگاه باید یک کانال مشترک را شریک شوند چه داستانی پیش می‌آید و چه پروتکل‌هایی برای کنترل دسترسی به کانال داریم، و در نهایت با مفاهیم پایه‌ی شبکه‌های محلی یا LAN آشنا شویم (مثل Ethernet و ایده‌های مرتبط با آن). در واقع این فصل می‌خواهد یک دید عملی هم بدهد که این تکنولوژی‌ها در دنیای واقعی چطور «پیاپی» می‌شوند.

اول از همه باید اصطلاحات را مرتب کنیم: در مسیر ارتباطی، میزبان‌ها و روترها را به‌عنوان node در نظر می‌گیریم و کانال‌هایی که دو node کنار هم را به هم وصل می‌کنند می‌شوند link. این link می‌تواند سیمی باشد، بی‌سیم باشد، یا داخل یک LAN تعریف شود. بسته‌ی لایه ۲ معمولاً به اسم frame شناخته می‌شود و کاری که می‌کند این است که یک datagram (همان خروجی لایه شبکه) را داخل خودش کپسوله می‌کند و روی یک لینک، از یک گره به گره فیزیکی مجاور منتقلش می‌کند. نکته‌ی کلیدی اینجا است که مسیر انتها-به-انتهای چند لینک تشکیل شده و ممکن است هر لینک، پروتکل Link layer مخصوص خودش را داشته باشد؛ مثلاً یک جا Ethernet باشد و یک جا 802.11، و هر کدام هم سرویس‌های متفاوتی بدهند.

حالا سرویس‌های Link layer را اگر بخواهیم «روان و ساده» جمع کنیم، مهم‌ترینش framing است: یعنی datagram را می‌گذاریم داخل frame و یک header و trailer هم اضافه می‌کنیم. داخل همین header معمولاً آدرس‌های MAC می‌آید که برای شناسایی مبدأ و مقصد روی همان لینک استفاده می‌شود و با آدرس IP فرق دارد؛ IP برای مسیریابی بین شبکه‌هاست ولی MAC بیشتر برای تحویل روی همان لینک/محیط محلی است. اگر رسانه مشترک باشد، بحث link access پیش می‌آید که یعنی چه کسی چه زمانی حق ارسال دارد. گاهی هم reliable delivery بین دو گره مجاور مطرح می‌شود (که ایده‌اش را از قبل شبیه سازوکارهای rdt می‌شناسیم)، ولی معمولاً روی لینک‌های کم‌خطا مثل فیبر خیلی جدی دنبال نمی‌شود؛ در عوض روی لینک‌های بی‌سیم که خطا بیشتر است، اهمیتش بالاتر می‌رود. علاوه بر این‌ها، flow control داریم که یعنی فرستنده و گیرنده‌ی دو سر لینک با هم هماهنگ می‌شوند که یکی

دیگری را خفه نکند و سرعت ارسال متناسب باشد. همچنین بحث error detection و error correction مطرح است: نویز و تضعیف سیگنال می تواند بیت ها را خراب کند، گیرنده سعی می کند خطا را تشخیص بدهد و یا درخواست ارسال مجدد کند یا frame را دور بیندازد، و در بعضی روش ها حتی خودش می تواند مقدار محدودی از خطا را اصلاح کند. در نهایت مفهوم half-duplex و full-duplex هم می آید: در half-duplex دو طرف می توانند ارسال کنند ولی هم زمان نه، اما در full-duplex این محدودیت کمتر/حذف می شود (بسته به تکنولوژی).

از نظر پیاده سازی هم این طور نیست که Link layer فقط یک «ایده» باشد؛ عملاً داخل هر میزبان یک adapter یا همان کارت شبکه/چیپ شبکه داریم که به آن NIC هم می گویند. این NIC به باس سیستم مثل PCI وصل می شود و ترکیبی از سخت افزار، نرم افزار و firmware است. سمت فرستنده، datagram را می گیرد، داخل frame می گذارد، بیت های کنترلی و چک خطا و اگر لازم شد سازوکارهای کنترل جریان/قابلیت اطمینان را اضافه می کند و می فرستد؛ سمت گیرنده هم frame را می گیرد، چک می کند خطا دارد یا نه، اگر لازم شد با مکانیزم های قابلیت اطمینان هماهنگ می شود، بعد datagram را بیرون می کشد و تحویل لایه های بالاتر می دهد.

بعد می رسیم به بحث تشخیص/اصلاح خطا. ایده ی پایه این است که مقداری بیت افزونه به داده اضافه کنیم که به آن ها EDC می گویند (بیت های Error Detection and Correction). داده ی اصلی که باید محافظت شود را می گذاریم D و با اضافه کردن افزونگی تلاش می کنیم خطا را بفهمیم. نکته ی مهم این است که تشخیص خطا صد درصد نیست، ولی معمولاً با زیادتر کردن افزونگی، احتمال از دست رفتن خطا خیلی کم می شود. یکی از ساده ترین روش ها parity است: single-bit parity که برای تشخیص خطاهای تکبیتی به کار می آید و two-dimensional parity که حتی می تواند در یک سناریوی مناسب، خطای تکبیتی را اصلاح هم بکند. یک یادآوری هم از Internet checksum داریم که بیشتر در لایه انتقال (مثلاً برای UDP) دیده می شود: فرستنده محتوا را به صورت عدد های ۱۶ بیتی جمع one's complement می زند و نتیجه را داخل فیلد چک سام می گذارد، گیرنده دوباره همان را حساب می کند و با مقدار دریافتی مقایسه می کند؛ اگر نخورد یعنی خطا تشخیص داده شده، اگر خورد یعنی «احتمالاً» درست است، ولی باز هم تضمین مطلق نیست.

روش قوی تر و خیلی رایج تر در عمل CRC یا Cyclic Redundancy Check است که در تکنولوژی هایی مثل Ethernet و 802.11 WiFi هم دیده می شود. ایده اش این است که بیت های داده را مثل یک عدد دودویی نگاه کنیم، یک الگوی مولد به اسم G (با طول $r+1$ بیت) انتخاب کنیم، و بعد r بیت CRC به اسم R بسازیم طوری که دنباله ی $D||R$ (یعنی الحاق داده و افزونگی) بر G در حساب پیمانه ای 2^r بخش پذیر شود. سمت گیرنده هم G را می داند، $D||R$ را تقسیم می کند؛ اگر باقیمانده صفر نبود، خیلی روشن می گوید خطا رخ داده است. مزیتش این است که توان تشخیصش بالاست و خطاهای مختلف (مثل خطاهای انفجاری تا طول مشخص) را با احتمال خیلی بالا می گیرد. مثال محاسباتی اسلاید هم دقیقاً می خواهد همین حس را بدهد که عملاً R همان باقیمانده ی تقسیم $GD \cdot 2^r$ است و به همین خاطر انتخاب R هوشمندانه باعث می شود $D||R$ دقیقاً بر G تقسیم شود.

بخش بعدی می‌رود سراغ لینک‌های چنددسترسی و پروتکل‌های MAC. از نظر نوع لینک، گاهی لینک point-to-point داریم (مثل PPP در سناریوهای قدیمی‌تر) که یک فرستنده و یک گیرنده‌اند و داستان ساده‌تر است؛ اما در بسیاری از محیط‌ها لینک broadcast یا محیط مشترک داریم (سیم یا بی‌سیم) که چند نود روی یک رسانه مشترک هستند. اینجا اگر دو یا چند نود هم‌زمان ارسال کنند، تداخل رخ می‌دهد و نتیجه‌اش collision است؛ یعنی گیرنده عملاً سیگنال‌های روی هم افتاده می‌گیرد و داده خراب می‌شود. پس باید یک پروتکل multiple access داشته باشیم که یک الگوریتم توزیع شده باشد و تعیین کند هر نود چه زمانی اجازه دارد ارسال کند، بدون اینکه یک کانال جداگانه برای هماهنگی داشته باشیم (چون خود هماهنگی هم باید روی همان کانال انجام شود!).

اگر بخواهیم یک پروتکل چنددسترسی ایده‌آل تصور کنیم، انتظار داریم وقتی فقط یک نود قصد ارسال دارد، بتواند با نرخ کامل کانال (مثلاً R بیت بر ثانیه) بفرستد، و وقتی M نود هم‌زمان داده دارند، هر کدام به‌طور متوسط سهم منصفانه‌ای نزدیک R/M بگیرند. همچنین دوست داریم سیستم کاملاً غیرمتمرکز باشد، هماهنگ‌کننده ویژه نداشته باشد، و به زمان‌بندی دقیق یا همگام‌سازی سراسری وابسته نباشد، و در عین حال ساده هم باشد.

در اسلایدها پروتکل‌های MAC به‌صورت کلی در سه خانواده دسته‌بندی می‌شوند. یک خانواده channel partitioning است که کانال را به تکه‌های کوچک‌تر تقسیم می‌کند (در زمان یا فرکانس یا گد) و به هر نود یک تکه اختصاص می‌دهد. TDMA نمونه معروف تقسیم زمانی است: زمان به دورها و شیارهای ثابت تقسیم می‌شود، هر ایستگاه در هر دور یک شیار ثابت دارد، و اگر داده نداشت، شیارش بی‌استفاده می‌ماند (که یعنی بهره‌وری ممکن است پایین بیاید ولی نظم خیلی خوب است). FDMA هم تقسیم فرکانسی است: طیف فرکانسی را به باندها تقسیم می‌کنیم، هر ایستگاه یک باند ثابت می‌گیرد، و اگر ارسال نکند آن باند همان‌طور بی‌کار می‌ماند.

خانواده دوم random access است: کانال را از قبل تقسیم نمی‌کنیم؛ هرکس هر وقت داده داشت با نرخ کامل ارسال می‌کند، و اگر چند نفر هم‌زمان زدند به کانال، collision رخ می‌دهد و بعد باید somehow از برخورد «ریکاوری» کنیم، مثلاً با ارسال مجدد با تأخیرهای کنترل شده. اینجا معمولاً دو سؤال مهم داریم: برخورد را چگونه تشخیص بدهیم و بعد از برخورد چگونه دوباره ارسال را مدیریت کنیم. نمونه‌های معروف این خانواده در اسلایدها slotted ALOHA، CSMA، ALOHA و نسخه‌های مهمش مثل CSMA/CD و CSMA/CA هستند که هر کدام ایده‌ی خاصی برای گوش دادن به کانال، تشخیص برخورد یا پرهیز از برخورد دارند.

مراجع